MindSpore Transformers

MindSpore Transformers 大模型系列课程

MindSpore Transformers LLM训练迁移与实践

主讲人：Jackey

昇思MindSpore研发工程师

MindSpore Transformers SIG Contributor

MindSpore Transformers 大模型系列课程

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

前置课程

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers LLM推理迁移与实践》

MindSpore Transformers

目录

1. Mcore架构介绍与模型搭建

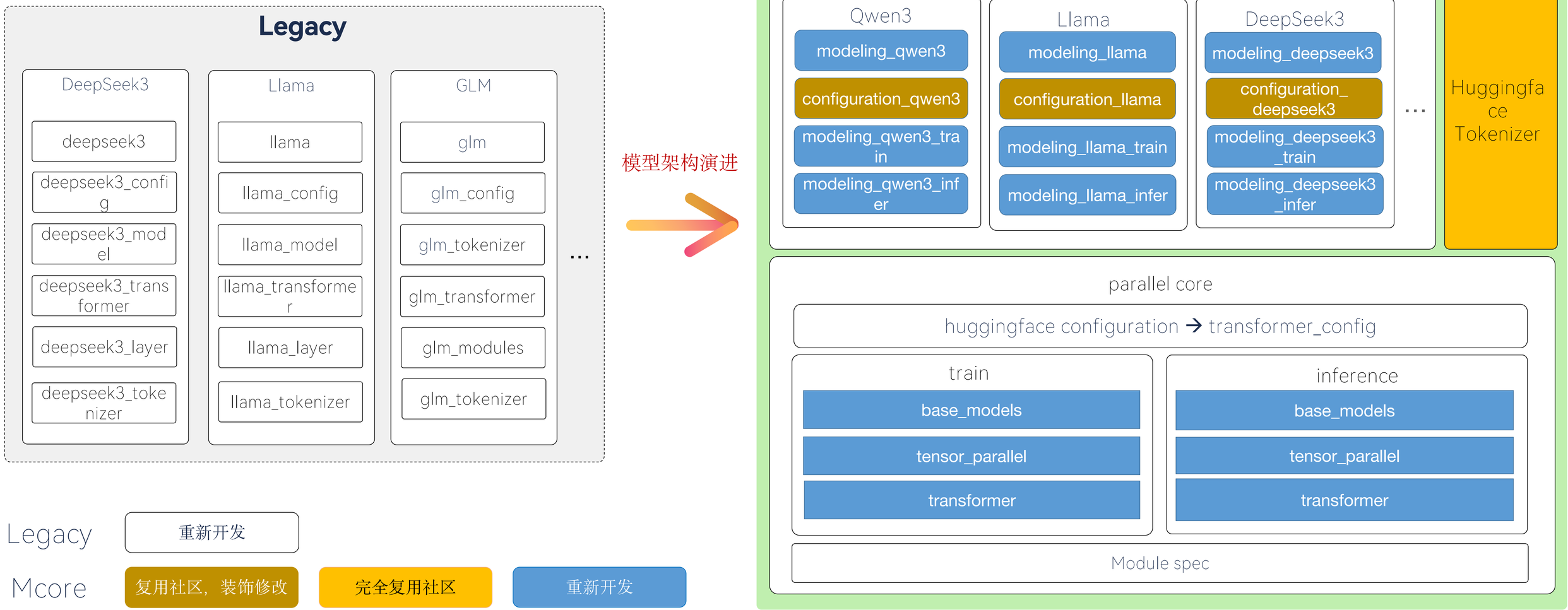
2. Hugging Face模型迁移

3. 拉起预训练任务

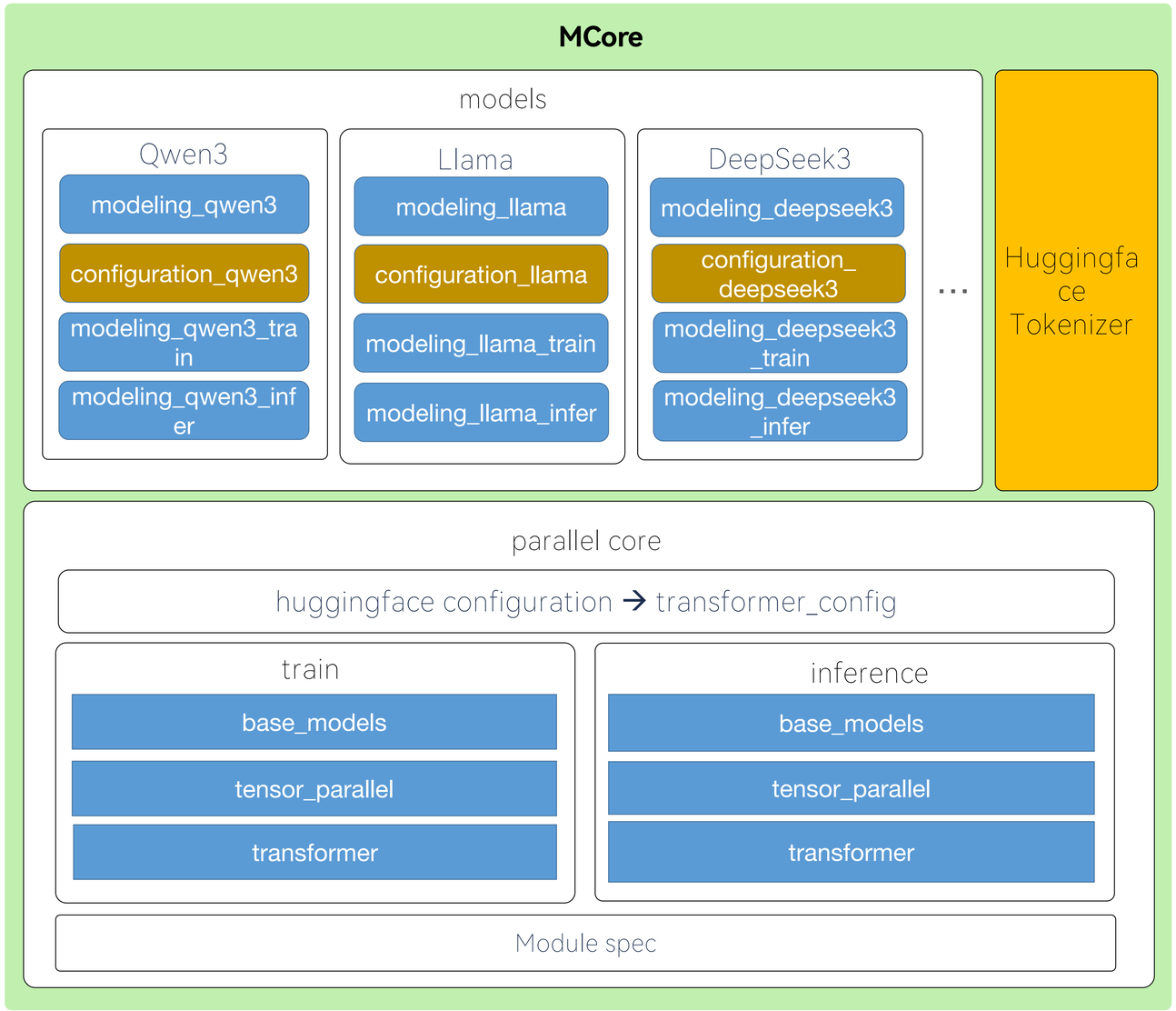
4. Hugging Face权重转换与微调任务

1. Mcore架构介绍与模型搭建

MindSpore Transformers Mcore模型库组件——整体架构



MindSpore Transformers Mcore模型库组件
——整体架构



1.提供**基础并行接口**和**ModuleSpec**模型搭建接口

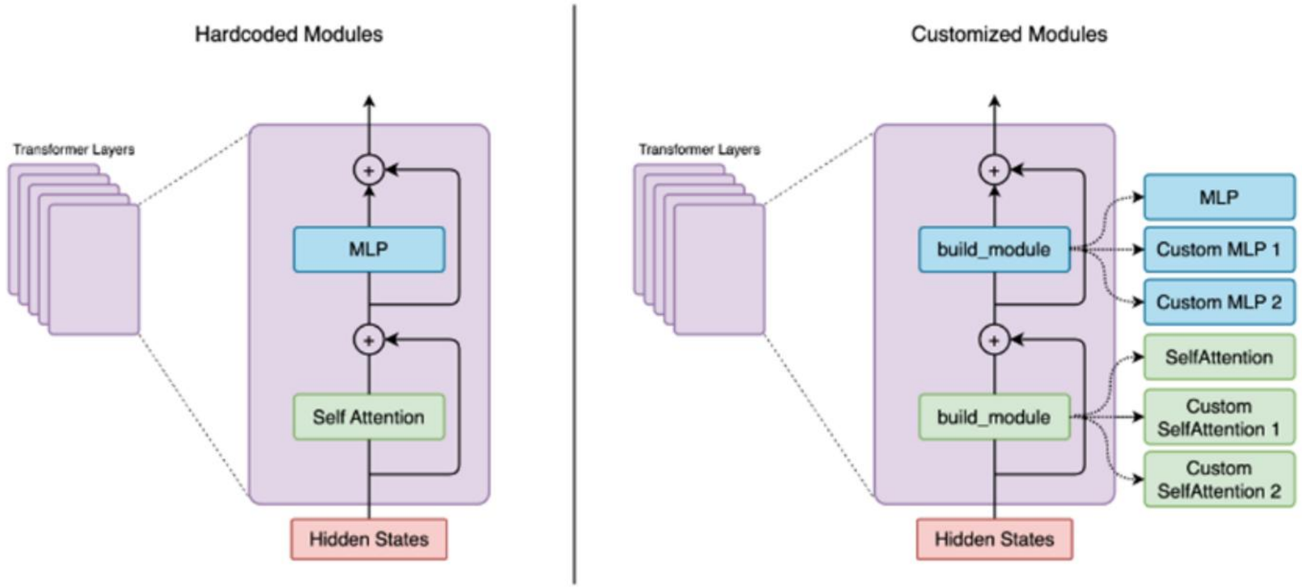
基础模块 + ModuleSpec --> 确定模型架构

2. 提供统一的模型配置接口TransformerConfig

TransformerConfig --> 确定模型配置

MindSpore Transformers Mcore模型库组件
——ModelSpec模型搭建

基于ModelSpec动态搭建模型




```
return ModuleSpec(
    module=mlp,
    submodules=MLPSubmodules(
        linear_fc1=ColumnParallelLinear,
        linear_fc2=RowParallelLinear,
    ),
)
```

```
return ModuleSpec(
    module=TransformerLayer,
    submodules=TransformerLayerSubmodules(
        input_layernorm=get_norm_cls(fused_norm),
        self_attention=ModuleSpec(
            module=SelfAttentionContiguous if use_contiguous_weight_layout_attention else SelfAttention,
            submodules=SelfAttentionSubmodules(
                linear_qkv=ColumnParallelLinear,
                core_attention=FlashAttention,
                linear_proj=RowParallelLinear,
                q_layernorm=get_norm_cls(fused_norm) if qk_layernorm else IdentityOp,
                k_layernorm=get_norm_cls(fused_norm) if qk_layernorm else IdentityOp,
            ),
        ),
        pre_mlp_layernorm=get_norm_cls(fused_norm),
        mlp=mlp
    )
)
```

MindSpore Transformers Mcore模型库组件
——公共ModuleSpec搭建接口

•get_gpt_layer_local_spec：构建单个 Transformer 层的模块规范，适用于每层结构相同的模型搭建（如Qwen3）

•get_gpt_decoder_block_spec：构建整个解码器块的模块规范，即多层Transformer模块，支持MoE、Dense混合层构建，适用于存在不同Transformer层结构的模型搭建（如DeepSeekV3）

•get_gpt_mtp_block_spec：根据已搭建的模块，生成mtp层的结构

```python
class TrainingQwen3ForCausalLM(Qwen3PreTrainedModel, TrainModelMixin):
    """
    Provide qwen2 model infer through network.

    Args:
        config (Qwen3Config): The config of qwen3 model.

    Returns:
        output: Tensor, the output of qwen3 decoderlayer

    """

    def __init__(self, config: Qwen3Config):
        super().__init__(config, auto_prefix=False)
        config: TransformerConfig = self.convert_to_transformer_config(self.config)

        self.model = GPTModel(
            config=config,
            transformer_layer_spec=get_gpt_layer_local_spec(
                qk_layernorm=True,
                use_contiguous_weight_layout_attention=config.use_contiguous_weight_layout_attention,
                use_interleaved_weight_layout_mlp=config.use_interleaved_weight_layout_mlp
            ),
            vocab_size=config.vocab_size,
            max_sequence_length=config.max_position_embeddings,
            position_embedding_type=config.position_embedding_type,
            rotary_base=self.config.rope_theta,
            share_embeddings_and_output_weights=self.config.tie_word_embeddings,
            post_process=self.config.post_process
        )
```

MindSpore Transformers Mcore模型库组件——GPTModel接口

GPTModel接口是所有类GPT结构模型的通用接口，也是我们迁移LLM模型用到的统一模型接口。

参数名	类型	描述	默认值
config	TransformerConfig	模型全局配置对象，包含隐藏层大小、注意力头数等核心参数	-
transformer_layer_spec	ModuleSpec	定义Transformer层结构的模块规范，指定SelfAttention/MLP等组件	-
vocab_size	int	词汇表大小，决定嵌入层和输出层的维度	-
max_sequence_length	int	序列最大长度，用于位置编码的初始化	-
share_embeddings_and_output_weights	bool	是否共享嵌入层与输出层权重	False
position_embedding_type	str	位置编码类型	'learned_absolute'
rotary_percent	float	应用旋转编码的维度比例	1.0
rotary_base	int	RoPE基础值	10000
rope_scaling	bool	是否启用RoPE外推算法	False
rope_scaling_factor	float	RoPE scaling factor	8.0
seq_len_interpolation_factor	float	序列长度插值因子	None
mtp_block_spec	ModuleSpec	MTP块规范	None

2. Hugging Face模型迁移

MindSpore Transformers Hugging Face模型迁移——文件结构

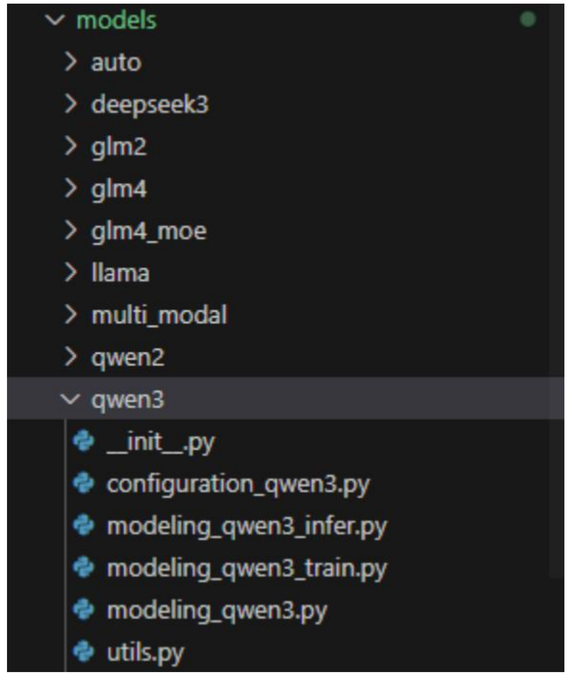
通用流程图

文件结构

模型代码放置于mindformers/models下，每个模型有独立文件夹

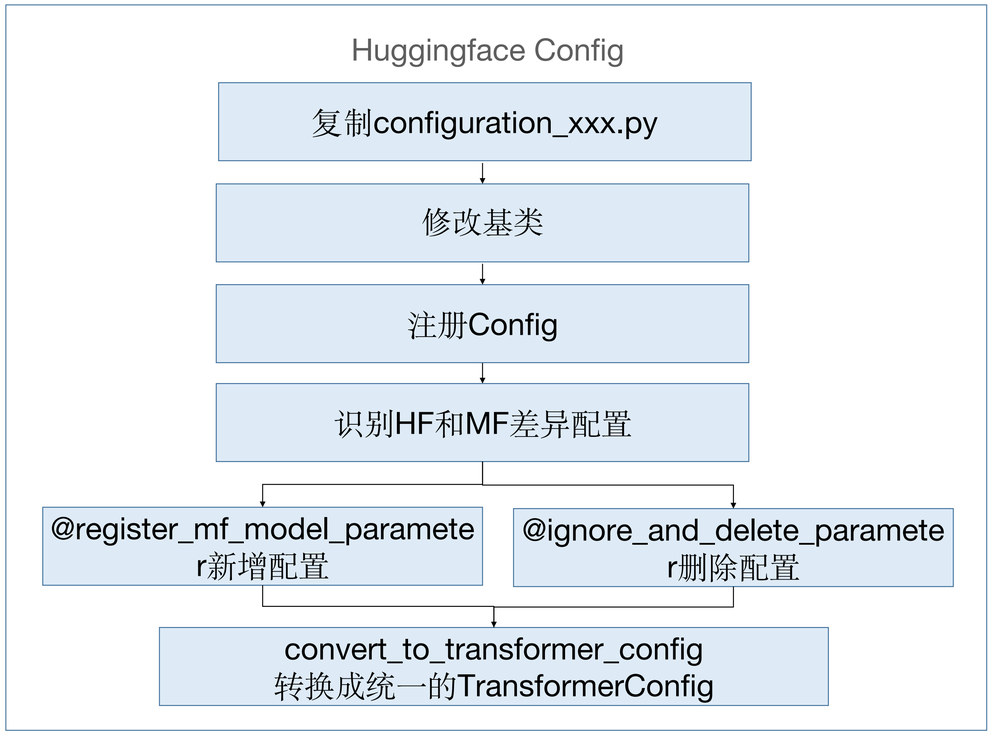
迁移HF模型时，主要需要创建以下文件：

- __init__.py
- configuration_xxx.py ---模型配置
- modeling_xxx.py ---模型类
- modeling_xxx_train.py ---训练模型类
- modeling_xxx_infer.py ---推理模型类（可选）
- utils.py



MindSpore Transformers Hugging Face模型迁移——HFConfig适配

1. 在configuration_xxx.py文件中，实现ModelConfig类。




```python
from mindformers.models.configuration_utils import PretrainedConfig
from mindformers.models.model_config_utils import (
    register_mf_model_parameter,
    ignore_and_delete_parameter,
    NotSupportedInfo
)
from mindformers.parallel_core.mf_model_config import MFModelConfig
from mindformers.tools.register import MindFormerRegister, MindFormerModuleType


zyw_hw, 3 months ago | 5 authors (sunyu-xuan and others)
@MindFormerRegister.register(MindFormerModuleType.CONFIG, legacy=False, search_names='qwen3')
class Qwen3Config(PretrainedConfig):
    """
    This is the configuration class to store the configuration of a [`Qwen3Model`]. It is used to instantiate a
    Qwen3 model according to the specified arguments, defining the model architecture. Instantiating a configuration
```

注册

修改基类

```python
@register_mf_model_parameter(
    mf_model_kwargs=MFModelConfig(
        pad_token_id=151643,
        block_size=32,
        num_blocks=1024,
        normalization='RMSNorm',
        add_bias_linear=False,
        gated_linear_unit=True,
        use_contiguous_weight_layout_attention=False
    ))
@ignore_and_delete_parameter(extra_ignore_param=[
    ('max_window_layers', NotSupportedInfo.useless),
    ('sliding_window', NotSupportedInfo.useless),
    ('use_sliding_window', NotSupportedInfo.useless),
    ('layer_types', NotSupportedInfo.useless),
])
def __init__(self,
             vocab_size=151936,
             hidden_size=4096,
             intermediate_size=22016,
             num_hidden_layers=32,
             num_attention_heads=32,
```

新增MF配置

删除配置

MindSpore Transformers Hugging Face模型迁移——HFConfig适配

支持的配置列表

配置名称	MF配置来源标签	类型|默认值|训练取值范围	类型|默认值|推理取值范围	归类	支持场景	计划	
num_layers	Megatron-LM			模型结构（ModelArch）	训练 推理	训练已验证 推理已验证	
hidden_size	Megatron-LM			模型结构（ModelArch）	训练 推理	训练已验证 推理已验证	
mtp_num_layers	Megatron-LM			模型结构（MTP）	训练	推理待支持	
mtp_loss_scaling_factor	Megatron-LM			模型结构（MTP）	训练		
num_attention_heads	Megatron-LM			模型结构（Attention）	训练 推理		
softmax_scale	Megatron-LM			模型结构（Attention） 模型结构（MLA）	训练 推理		
num_query_groups	Megatron-LM			模型结构（Attention）	训练 推理		
ffn_hidden_size	Megatron-LM			模型结构（MLP）	训练 推理		
kv_channels	Megatron-LM			模型结构（Attention） 模型结构（Embedding）	训练 推理		
hidden_dropout	Megatron-LM			模型结构（ModelArch）	训练 推理		
attention_dropout	Megatron-LM			模型结构（Attention）	训练		
fp32_residual_connection	Megatron-LM			模型结构（ModelArch）	不支持	待确认	
apply_residual_connection_post_layernorm	Megatron-LM			模型结构（ModelArch）	训练 推理		
layernorm_epsilon	Megatron-LM			模型结构（Norm）	训练 推理		
layernorm_zero_centered_gamma	Megatron-LM			模型结构（Norm）	不支持	待确认	训练支持
add_qkv_bias	Megatron-LM			模型结构（Attention）	训练 推理		
activation_func	Megatron-LM			模型结构（ModelArch）	训练 推理		
num_moe_experts	Megatron-LM			模型结构（MOE）	训练 推理		

文档：MindFormers-LLM配置白名单管理表格

代码： mindformers/parallel_core/transformer_config.py

MindSpore Transformers Hugging Face模型迁移
——HFConfig适配

使用时：转换成统一的**TransformerConfig**

```python
class TrainingQwen3ForCausalLM(Qwen3PreTrainedModel, TrainModelMixin):
    """
    Provide qwen2 model infer through network.

    Args:
        config (Qwen3Config): The config of qwen3 model.

    Returns:
        output: Tensor, the output of qwen3 decoderlayer

    """

    def __init__(self, config: Qwen3Config):
        super().__init__(config, auto_prefix=False)
        config: TransformerConfig = self.convert_to_transformer_config(self.config)

        self.model = GPTModel(
            config=config,
            transformer_layer_spec=get_gpt_layer_local_spec(
                qk_layernorm=True,
                use_contiguous_weight_layout_attention=config.use_contiguous_weight_layout_attention,
                use_interleaved_weight_layout_mlp=config.use_interleaved_weight_layout_mlp
            ),
            vocab_size=config.vocab_size,
            max_sequence_length=config.max_position_embeddings,
            position_embedding_type=config.position_embedding_type,
            rotary_base=self.config.rope_theta,
            share_embeddings_and_output_weights=self.config.tie_word_embeddings,
            post_process=self.config.post_process
        )
```

转换config

初始化GPTModel

convert_to_transformer_config()

入参介绍：

model_config(PretrainedConfig):
模型Config

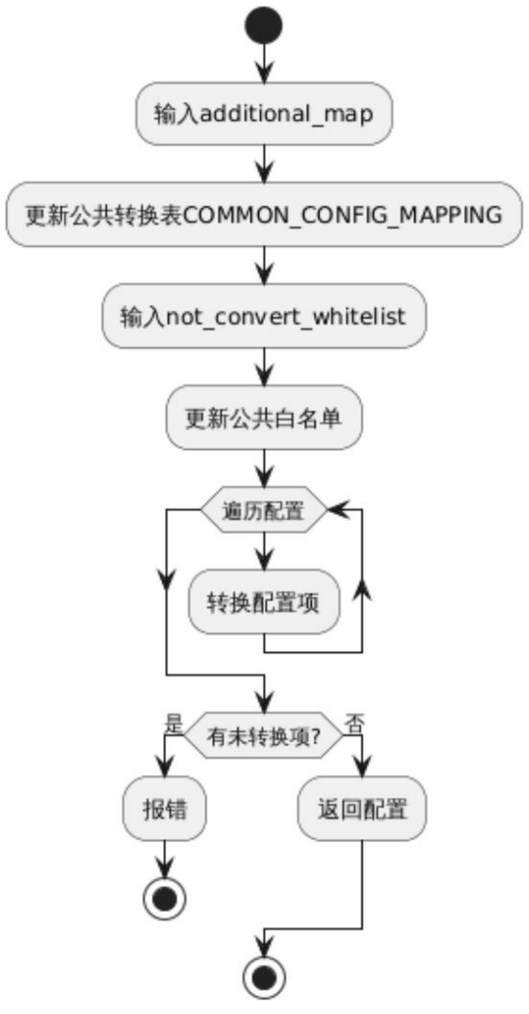
is_mla_model(bool):
是否是使用MLA模块的模型

additional_map (dict):
可自定义转换配置，格式为
{“source_key0”: (“target_key0”, func), {“source_key1”: “target_key1”}

not_convert_whitelist (set):
无需转换的配置，传入一个集合。

MindSpore Transformers Hugging Face模型迁移——HFConfig适配

convert_to_transformer_config()的工作流



公共转换表 COMMON_CONFIG_MAPPING

位置:
mindformers\parallel_core\transformer_config_utils.py

```
# Model Architecture
("mtp_depth", "num_nextn_predict_layers", "mtp_num_layers"): "mtp_num_layers",
("mtp_loss_factor", "mtp_loss_scaling_factor"): "mtp_loss_scaling_factor",
("num_heads", "num_attention_heads", "n_head"): "num_attention_heads",
("n_kv_heads", "num_key_value_heads", "num_query_groups"): "num_query_groups",
("intermediate_size", "ffn_hidden_size"): "ffn_hidden_size",
("head_dim", "kv_channels"): "kv_channels",
```

使用示例

```
"""
假设XXXConfig存在特殊配置。

覆盖公共转换：mtp_layers, compute_in_fp32

无用配置：kv_cache, top_k
"""

def is_fp32(compute_in_fp32):
  if compute_in_fp32:
    return float32
  return bfloat16

class TrainingXXXForCausalLM(TrainModelMixin, XXXPreTrainedModel):
    def __init__(self, config: XXXConfig):
        super().__init__(config, auto_prefix=False)

        transformer_config = self.convert_to_transformer_config(config,
                                                                 is_mla_model=True,
                                                                 additional_map={"mtp_layers": "mtp_num_layers",
                                                                                 "compute_in_fp32": ("compute_dtype", is_fp32)
                                                                                 },
                                                                 not_convert_whitelist={"kv_cache", "top_k"}
                                                                )
```

MindSpore Transformers Hugging Face模型迁移——HFConfig适配

convert_to_transformer_config()常见报错

① 遗漏配置未转换

```
if not_convert_keys_list:
    raise ValueError(f"Keys: {not_convert_keys_list} dose not be converted! "
                     f"Please check your config parameters.")
```

原因：存在配置未发生转换
解决方案：若不需要该配置，则加到not_convert_whitelist中；若需要，则加到additional_map中。

② 转换发生冲突

```
if mapping_key in update_dict:
    raise KeyError(f"Multiple configurations provided for the same setting. "
                   f"Please check these conflicting configs: {list(reversed_mapping[mapping_key])}")
```

原因：存在多个配置转换到了同一个TransformerConfig配置
解决方案：检查additional_map逻辑，是否和公共映射同时映射了同一个配置，并修改。

MindSpore Transformers Hugging Face模型迁移——模型适配

2. 在utils.py文件中，实现模型基类。

①实现模型基类xxxPretrainedModel：继承PretrainedModel和ModelMixin类

② 指定config_class属性为模型对应Config类

```python
"""Qwen3 models' utils."""
from mindformers.models.qwen3.configuration_qwen3 import Qwen3Config
from mindformers.models.modeling_utils import PreTrainedModel
from mindformers.parallel_core.utils.model_mixin import ModelMixin
      zxq, 5 months ago • 【dev】【inference】qwen3_mcore模型适配

zxq, 3 months ago | 1 author (zxq)
class Qwen3PreTrainedModel(PreTrainedModel, ModelMixin):
    """
    An abstract class to handle weights initialization and a simple interface for downloading and loading pretrained
    models.
    """

    config_class = Qwen3Config
    base_model_prefix = "Qwen3"
```

继承基类

指定config类

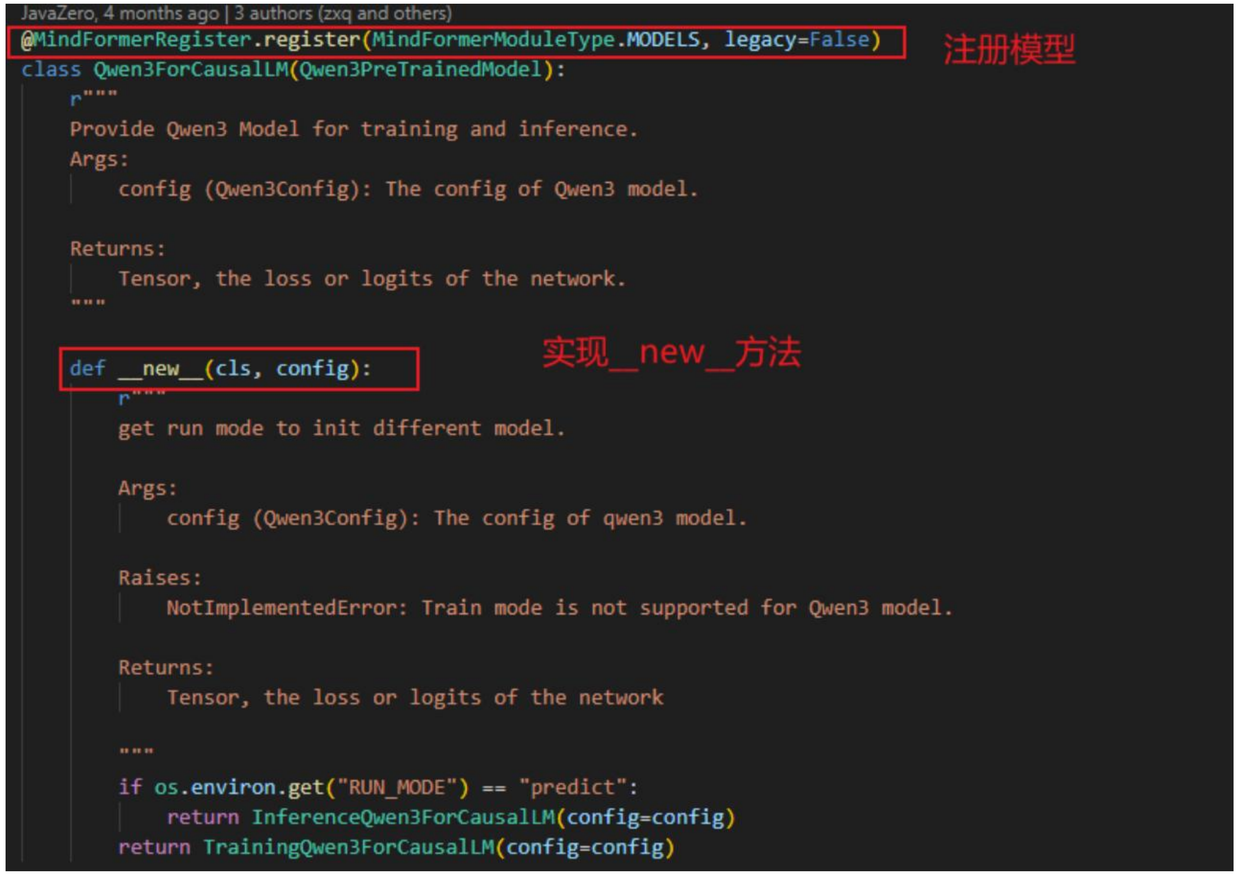
MindSpore Transformers Hugging Face模型迁移——模型适配

2. 在modeling_xxx.py文件中，实现模型类。

① 实现最外层模型类：继承xxxPretrainedModel

② 注册模型

③ 实现__new__方法，根据不同的RUN_MODE，返回不同的模型



MindSpore Transformers Hugging Face模型迁移——模型适配

3. 在**modeling_xxx_train.py**文件中，实现训练模型类。

```python
class TrainingQwen3ForCausalLM(Qwen3PreTrainedModel, TrainModelMixin):
    """
    Provide qwen2 model infer through network.

    Args:
        config (Qwen3Config): The config of qwen3 model.

    Returns:
        output: Tensor, the output of qwen3 decoderlayer

    """

    def __init__(self, config: Qwen3Config):
        super().__init__(config, auto_prefix=False)
        config: TransformerConfig = self.convert_to_transformer_config(self.config)

        self.model = GPTModel(
            config=config,
            transformer_layer_spec=get_gpt_layer_local_spec(
                qk_layernorm=True,
                use_contiguous_weight_layout_attention=config.use_contiguous_weight_layout_attention,
                use_interleaved_weight_layout_mlp=config.use_interleaved_weight_layout_mlp
            ),
            vocab_size=config.vocab_size,
            max_sequence_length=config.max_position_embeddings,
            position_embedding_type=config.position_embedding_type,
            rotary_base=self.config.rope_theta,
            share_embeddings_and_output_weights=self.config.tie_word_embeddings,
            post_process=self.config.post_process
        )
```

转换config

初始化GPTModel

① **实现训练模型类**：继承xxxPretrainedModel和训练模型基类TrainModelMixin

② **实现__init__方法**：方法内部将config转换为TransformerConfig，然后初始化GPTModel

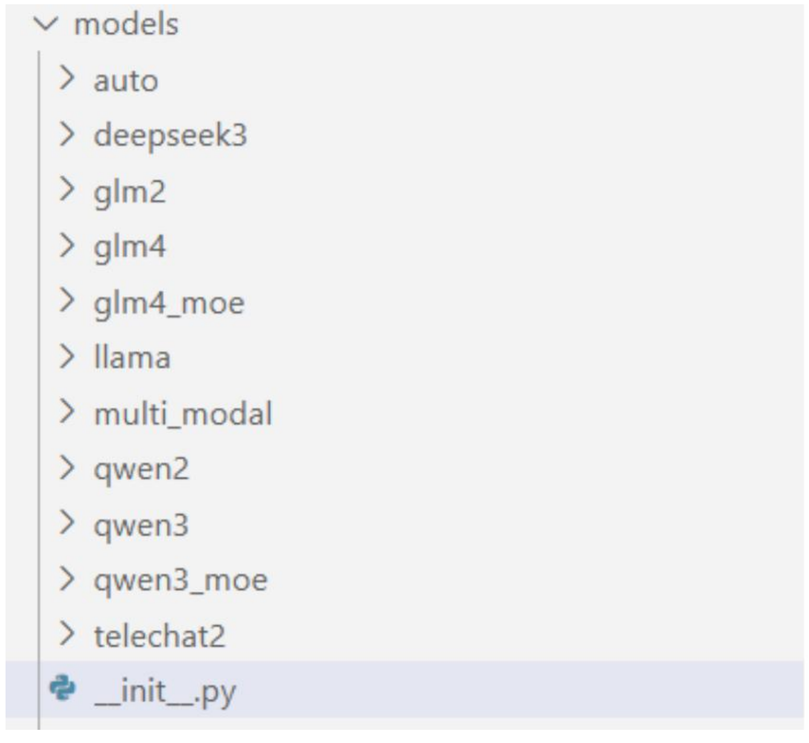
③ **实现construct方法**：复用GPTModel的construct()方法即可

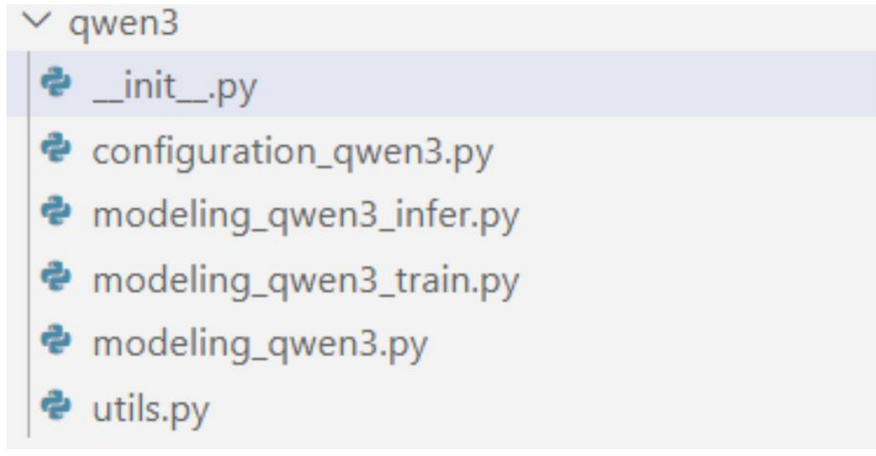
```python
def construct(
        self,
        input_ids: Tensor,
        position_ids: Tensor = None,
        attention_mask: Tensor = None,
        decoder_input: Tensor = None,
        labels: Tensor = None,
        extra_block_kwargs=None,
        prefix_keys_values=None,
        loss_mask=None,
        actual_seq_len=None
):
    """Qwen3 construct for training"""
    return self.model(
        input_ids=input_ids,
        position_ids=position_ids,
        attention_mask=attention_mask,
        decoder_input=decoder_input,
        labels=labels,
        extra_block_kwargs=extra_block_kwargs,
        prefix_keys_values=prefix_keys_values,
        loss_mask=loss_mask,
        actual_seq_len=actual_seq_len
    )
```

MindSpore Transformers Hugging Face模型迁移——模型适配

5. 其他注意事项

为了确保register正确生效，需要在__init__代码中修改import，确保import mindformers时，模型已经正确注册。






```python
from .qwen3 import (
    Qwen3Config,
    Qwen3PreTrainedModel,
    Qwen3ForCausalLM,
)
```

```python
from .utils import Qwen3PreTrainedModel
from .configuration_qwen3 import Qwen3Config
from .modeling_qwen3 import Qwen3ForCausalLM
from .modeling_qwen3_infer import InferenceQwen3ForCausalLM
from .modeling_qwen3_train import TrainingQwen3ForCausalLM

__all__ = [
    "Qwen3Config",
    "Qwen3ForCausalLM",
    "InferenceQwen3ForCausalLM",
    "TrainingQwen3ForCausalLM",
    "Qwen3PreTrainedModel",
]
```

3. 拉起预训练任务

MindSpore Transformers 拉起预训练任务
——创建Yaml文件

模块名称	模块用途
基础配置	基础配置主要用于指定 MindSpore 随机种子以及加载权重的相关设置。
数据集配置	数据集配置主要用于 MindSpore 模型训练时的数据集相关设置。详情可参考 数据集 。
模型配置	不同的模型配置参数存在差异，模板中的参数为通用配置。
模型优化配置	MindSpore Transformers 提供重计算相关配置，以降低模型在训练时的内存占用，详情可参考 重计算 。
模型训练配置	启动模型训练时相关参数的配置模块，模板中主要包含 trainer、runner_config、runner_wrapper、学习率（lr_schedule）以及优化器（optimizer）相关训练所需模块的参数。
并行配置	为了提升模型的性能，在大规模集群的使用场景中通常需要为模型配置并行策略，详情可参考 分布式并行 。
回调函数配置	MindSpore Transformers 提供封装后的 Callbacks 函数类，主要实现在模型训练过程中返回模型的训练状态并输出、保存模型权重文件等一些操作，目前支持以下几个 Callbacks 函数类。 1. MFLossMonitor 该回调函数类主要用于在训练过程中对训练进度、模型 Loss、学习率等信息进行打印 2. SummaryMonitor 该回调函数类主要用于收集 Summary 数据，详情可参考 mindspore.SummaryCollector 。 3. CheckpointMonitor 该回调函数类主要用于在模型训练过程中保存模型权重文件。
context 配置	Context 配置主要用于指定 mindspore.set_context 中的相关参数。
性能分析工具配置	MindSpore Transformers 提供 Profile 作为模型性能调优的主要工具，详情可参考 性能调优指南 。

MindSpore Transformers 拉起预训练任务
——创建Yaml文件

1. 基础配置修改

配置项	值	说明
use_legacy	False	指定是否使用mcore模型
pretrained_model_dir	"path/to/model_dir"	Hugging Face模型配置所在的目录路径
seed	0	设置全局随机种子
output_dir	'./output'	设置日志、检查点、策略等文件保存路径
load_checkpoint	''	加载权重的文件或文件夹路径
auto_trans_ckpt	False	如果为true，自动转换load_checkpoint以在分布式模型中加载
resume_training	False	启用断点续训功能
run_mode	‘train'	设置模型的运行模式：train、finetune 或 predict
use_parallel	True	启用并行模式
load_ckpt_format	'safetensors'	加载检查点的格式，可以是 ckpt 或 safetensors

```yaml
use_legacy: False                                                  # Specifies whether to use the mcore model.
# pretrained_model_dir: &pretrained_model_dir "path/to/model_dir"  # The directory path where the Hugging Face model configuration is located.
seed: 0                                                            # Set the global seed.
output_dir: './output'                                             # Set the path where log, checkpoint, strategy, etc. files are saved
load_checkpoint: ''                                                # File or folder paths for loading weights
auto_trans_ckpt: False                                             # If true, auto transform load_checkpoint to load in distributed model
resume_training: False                                             # Enable resumable training after breakpoint.
run_mode: 'train'                                                  # Set the running mode of the model: `train`, `finetune` or `predict`
use_parallel: True                                                 # Enable parallel mode
load_ckpt_format: 'safetensors'                                    # The format of loading checkpoint, either `ckpt` or `safetensors`
```

MindSpore Transformers 拉起预训练任务
——创建Yaml文件

2. 数据集配置修改

使用Megatron数据集进行预训练

```yaml
train_dataset: &train_dataset
  data_loader:
    type: BlendedMegatronDatasetDataLoader
    datasets_type: "GPTDataset"
    sizes:
      - 400  # Number of training set data samples.
      - 0    # Number of test set data samples. Currently, configuration is not supported.
      - 0    # Number of eval set data samples. Currently, configuration is not supported.
    config:  # GPTDataset Configs
      seed: 1234                        # Data sampling random seed
      split: "1, 0, 0"                  # The usage ratio of training, testing and evaluation sets. Currently, configuration is not supported.
      seq_length: 4096                  # The sequence length of the data set returned.
      eod_mask_loss: False              # Whether to calculate loss at eod.
      reset_position_ids: False         # Whether to reset position_ids at eod.
      create_attention_mask: True       # Whether to return `attention_mask`.
      reset_attention_mask: False       # Whether to reset the `attention_mask` at eod and return a stepped `attention_mask`.
      create_compressed_eod_mask: False # Whether to return the compressed `attention_mask`.
      eod_pad_length: 128               # Set the length of the compressed `attention_mask`.
      eod: 1                            # The token id of `eod` in the dataset.
      pad: -1                           # The token id of `pad` in the dataset.
      data_path:  # Megatron dataset sampling ratio and path.
        - '0.3'                         # 数据集1的占比
        - "/path/megatron_data1"        # 数据集1的bin文件路径（去除.bin后缀）
        - '0.7'                         # 数据集2的占比
        - "/path/megatron_data2"        # 数据集2的bin文件路径（去除.bin后缀）
  input_columns: ["input_ids", "labels", "loss_mask", "position_ids", "attention_mask"]
  construct_args_key: ["input_ids", "labels", "loss_mask", "position_ids", "attention_mask"]
  num_parallel_workers: 8
  python_multiprocessing: False
  drop_remainder: True
  numa_enable: False
  prefetch_size: 1
  seed: 1234
```

数据集 | MindSpore Transformers master 文档 | 昇思MindSpore社区

MindSpore Transformers 拉起预训练任务
——创建Yaml文件

3. 模型配置修改

1.**HF配置**：模型配置与Hugging Face配置保持一致

2.**MF配置**：添加MF新增配置

```python
@register_mf_model_parameter(
    mf_model_kwargs=MFModelConfig(
        pad_token_id=151643,
        block_size=32,
        num_blocks=1024,
        normalization='RMSNorm',
        add_bias_linear=False,
        gated_linear_unit=True,
        use_contiguous_weight_layout_attention=False
    ))
@ignore_and_delete_parameter(extra_ignore_param=[
    ('max_window_layers', NotSupportedInfo.useless),
    ('sliding_window', NotSupportedInfo.useless),
    ('use_sliding_window', NotSupportedInfo.useless),
    ('layer_types', NotSupportedInfo.useless),
])
def __init__(self,
             vocab_size=151936,
             hidden_size=4096,
             intermediate_size=22016,
             num_hidden_layers=32,
             num_attention_heads=32,
```

新增MF配置

删除配置

HFConfig迁移示例

```yaml
model:
  model_config:
    # hf config
    model_type: "qwen3"
    architectures: ["Qwen3ForCausalLM"]
    vocab_size: 51200                   # Vocabulary size of the model
    hidden_size: 12288                  # Dimension of hidden layer
    intermediate_size: 49152             # Dimension of intermediate layer
    num_hidden_layers: 24               # Number of hidden layers
    num_attention_heads: 96             # Number of attention heads
    # ...
    # mf config
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    # ...
```

指定模型Config和模型

yaml配置示例

MindSpore Transformers 拉起训练任务
——创建Yaml文件

4. 进阶配置修改

优化器

```yaml
# optimizer
optimizer:
  type: AdamW              # Set the optimizer class, the optimizer is mainly used to calculate the gradient for model training
  betas: [0.9, 0.999]      # The exponential decay rate of `moment1` and `moment2`. Each parameter range (0.0, 1.0)
  eps: 1.e-6               # Add it to the denominator to improve numerical stability. Must be greater than 0
  weight_decay: 0.01       # Set the optimizer weight decay coefficient

# lr schedule
lr_schedule:
  type: CosineWithWarmUpLR    # Set the lr_schedule class
  learning_rate: 1.e-6        # Set the initialized learning rate size
  lr_end: 1.e-6               # Final value of the learning rate
  warmup_ratio: 0             # Ratio of warmup phase to total training steps
  total_steps: -1             # -1 means it will load the total steps of the dataset
```

学习率

并行配置

```yaml
parallel_config:
  data_parallel: &dp 8          # Set the number of data parallel
  model_parallel: 1             # Set the number of model parallel
  pipeline_stage: 1             # Set the number of pipeline parallel
  context_parallel: 1           # Set the number of sequence parallel
  use_seq_parallel: True        # Corresponding to Megatron Short Sequence Parallelism
  micro_batch_num: 1            # Set the pipeline parallel microbatch size.
```

batch_size

```yaml
runner_config:
  epochs: 2       # Set the number of rounds for model training
  batch_size: 1   # Set the sample size of the batch data
```

MindSpore Transformers 拉起训练任务
——拉起任务

单机多卡启动

在 MindSpore Transformers 代码根目录下执行 msrun 启动脚本，进行单机预训练。命令中的路径需替换为真实路径。需要在 yaml 中修改适配单机的并行策略和模型层数。

```
bash scripts/msrun_launcher.sh "run_mindformer.py \
--config configs/qwen3/pretrain_qwen3_32b_4k.yaml \
--auto_trans_ckpt False \
--use_parallel True \
 --run_mode train" 8
```

多机多卡启动

- 机器 1 IP: 192.168.1.1 （作为主节点）

```
# 机器 1 的启动指令
master_ip=192.168.1.1
node_rank=0

cd $MINDFORMERS_HOME
bash scripts/msrun_launcher.sh "run_mindformer.py \
--config configs/qwen3/pretrain_qwen3_32b_4k.yaml \
--auto_trans_ckpt False \
--use_parallel True \
--run_mode train" \
16 8 $master_ip 8118 $node_rank output/msrun_log False
7200
```

- 机器 2 IP: 192.168.1.2

```
# 机器 2 的启动指令
master_ip=192.168.1.1
node_rank=1

cd $MINDFORMERS_HOME
bash scripts/msrun_launcher.sh "run_mindformer.py \
--config configs/qwen3/pretrain_qwen3_32b_4k.yaml \
--auto_trans_ckpt False \
--use_parallel True \
--run_mode train" \
16 8 $master_ip 8118 $node_rank output/msrun_log False 7200
```

4. Hugging Face权重转换与微调任务

Hugging Face 权重转 MindSpore Transformers 权重

① 加载权重	② 模型训练	③ 保存权重		④ 后续使用
随机初始化权重	预训练		自动切分合并	继续训练、后训练
加载MindSpore Safetensors	自动切分	继续训练	检查点保存　训练结束保存	离线合并	推理、部署、评测
加载Hugging Face Safetensors	离线转换 在线加载	微调		离线合并 权重反转	支持Hugging Face生态使用

Hugging Face 权重转 MindSpore Transformers 权重

➢ 离线转换 Hugging Face 权重

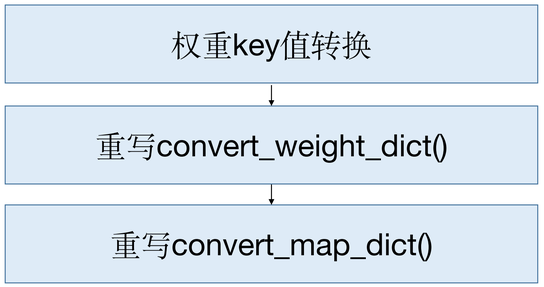
对于规格较大的模型，为了方便使用，MindSpore Transformers 推荐对其进行离线转换后再加载训练的操作，通过库上对应模型的离线转换脚本，对 Hugging Face 权重进行离线转换。现支持模型： DeepSeek-V3。

```python
for layer_id in range(total_num_layers):
    # Dequant weights firstly
    hf_layer_weights = dequant_layer_weights(layer_id, hf_layer_weights)
    # Get the replaced weight name and corresponding value in ms
    ms_layer_weights = defaultdict()
    # pre/post-process of the DeepSeekV3 model weight
    if layer_id == 0:
        ms_layer_weights.update(_model_preprocess_hf_to_ms(hf_layer_weights))
    if layer_id == total_num_layers - 1:
        ms_layer_weights.update(
            _model_postprocess_hf_to_ms(hf_layer_weights, has_mtp_layers)
        )
    # MTP Layers process
    if layer_id > num_layers - 1:
        ms_layer_weights.update(_mtp_hf_to_ms(layer_id, hf_layer_weights, config))
    # no MTP layers process
    else:
        # MLA Layers process
        ms_layer_weights.update(
            _trans_model_layer_norm_hf_to_ms(hf_layer_weights, layer_id)
        )
        ms_layer_weights.update(
            _trans_model_layer_attn_hf_to_ms(hf_layer_weights, layer_id)
        )
        # MLP Layers process
        ms_layer_weights.update(
            _trans_model_layer_mlp_hf_to_ms(hf_layer_weights, config, layer_id)
        )
```

脚本代码节选

Hugging Face 权重转 MindSpore Transformers 权重

➢ 在线加载 Hugging Face 权重



1.权重key值转换
编写weight_mapping属性。

功能：读取safetensors权重后，修改其key值

```python
class Qwen3PreTrainedModel(PreTrainedModel, ModelMixin):
    """
    An abstract class to handle weights initialization and a simple interface for downloading and loading pretrained
    models.
    """

    config_class = Qwen3Config
    base_model_prefix = "Qwen3"

    weight_mapping = [
        ('model.embed_tokens.', 'embedding.word_embeddings.'),
        ('.self_attn.q_proj.', '.self_attention.linear_q.'),
        ('.self_attn.k_proj.', '.self_attention.linear_k.'),
        ('.self_attn.v_proj.', '.self_attention.linear_v.'),
        ('.self_attn.o_proj.', '.self_attention.linear_proj.'),
        ('.self_attn.q_norm.', '.self_attention.q_layernorm.'),
        ('.self_attn.k_norm.', '.self_attention.k_layernorm.'),
        ('.mlp.gate_proj.', '.mlp.gating.'),
        ('.mlp.down_proj.', '.mlp.linear_fc2.'),
        ('.mlp.up_proj.', '.mlp.hidden.'),
        ('.post_attention_layernorm.', '.pre_mlp_layernorm.'),
        ('model.norm.', 'decoder.final_layernorm.'),
        ('lm_head.', 'output_layer.'),
        ('model.layers.', 'decoder.layers.')
    ]
```

Hugging Face 权重转 MindSpore Transformers 权重

➢ 在线加载 Hugging Face 权重

2.重写convert_weight_dict()

在训练模型类中，重写此方法。

功能：读取safetensors权重后，修改其value值

```python
def convert_weight_dict(self, source_dict, **kwargs):
    ...

    # Concat QKV weight
    if use_contiguous_weight_layout_attention:
        self.concat_qkv_weight_infer(wq_keys, wk_keys, wv_keys,
                                     qkv_dict, condition, ms_weight_dict)
    else:
        self.concat_qkv_weight_megatron(
            wq_keys=wq_keys, wk_keys=wk_keys, wv_keys=wv_keys,
            qkv_weight_dict=qkv_dict, condition=condition,
            ms_weight_dict=ms_weight_dict,
            head_dim=transformer_config.kv_channels,
            n_kv_heads=transformer_config.num_query_groups,
            num_attention_heads=transformer_config.num_attention_heads)

    # Concat FFN weight
    if use_interleaved_weight_layout_mlp:
        self.concat_ffn_weight_megatron(
            w1_keys=w1_keys, w3_keys=w3_keys,
            ffn_weight_dict=qkv_dict, condition=condition,
            ms_weight_dict=ms_weight_dict,
            ffn_hidden_size=transformer_config.ffn_hidden_size
        )
    else:
        self.concat_ffn_weight_infer(w1_keys, w3_keys,
                                     qkv_dict, condition, ms_weight_dict)

    return ms_weight_dict
```

Qwen3 处理逻辑节选

HuggingFace权重

Qwen3 在线加载逻辑

权重 key 是否和模型网络一致

是

否

处理 Hugging Face 权重

key、value 转换映射

linear_q、k、v 离散排布拼接

gate、up_proj 离散排布拼接

落盘 MindSpore Transformers 权重

识别为MindSpore Transformers权重，加载并自动切分

模型训练

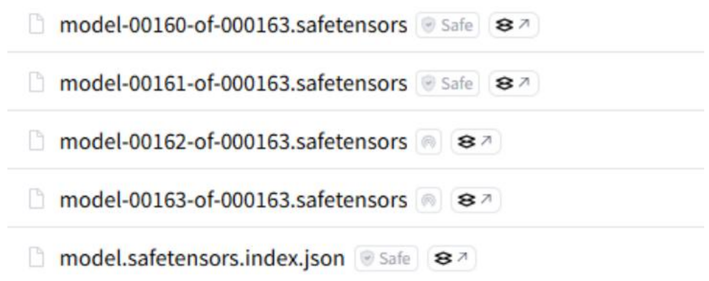
Hugging Face 权重转 MindSpore Transformers 权重

➢ 在线加载 Hugging Face 权重

3.重写convert_map_dict()
在训练模型类中，重写此方法。

功能：读取safetensors权重的json文件后，修改其key, value值

原因：safetensors权重，除了safetensors文件之外，还有index.json文件。需要进行同步修改。



hf safetensors 文件示例

```python
def convert_map_dict(self, hf_name_map_dict, **kwargs):
    ms_name_map_dict = {}
    wq_keys = []
    w1_keys = []

    # Get Q and gate keys
    for k, v in hf_name_map_dict.items():
        k = self.convert_name(k)
        ms_name_map_dict.update({k: v})
        part = k.split('.')
        if part[-2] == 'linear_q':
            wq_keys.append(k)
        if part[-2] == 'gating':
            w1_keys.append(k)

    # Only keep the key of Q to correspond to the result after QKV concat
    for wq_key in wq_keys:
        wk_key = wq_key.replace('linear_q', 'linear_k')
        wv_key = wq_key.replace('linear_q', 'linear_v')
        wq_value = ms_name_map_dict.pop(wq_key)
        ms_name_map_dict.pop(wk_key)
        ms_name_map_dict.pop(wv_key)

        w_qkv_key = wq_key.replace('linear_q', 'linear_qkv')
        w_qkv_value = wq_value
        ms_name_map_dict.update({w_qkv_key: w_qkv_value})

    # Only keep the key of gating to correspond to the result after FFN concat
    for w1_key in w1_keys:
        w3_key = w1_key.replace('gating', 'hidden')
        w1_value = ms_name_map_dict.pop(w1_key)
        ms_name_map_dict.pop(w3_key)

        w_gate_hidden_key = w1_key.replace('gating', 'linear_fc1')
        w_gate_hidden_value = w1_value
        ms_name_map_dict.update({w_gate_hidden_key: w_gate_hidden_value})

    return ms_name_map_dict
```

Qwen3 **convert_map_dict()**

MindSpore Transformers 拉起微调任务
——微调任务

微调任务与预训练任务的区别有：

1. **加载配置与权重**：需要加载来自Hugging Face的模型的配置和权重
2. **使用微调数据集**：一般微调使用Hugging Face数据集

1. 基础配置修改

配置项	值	说明
use_legacy	False	指定是否使用mcore模型
pretrained_model_dir	"path/to/model_dir"	Hugging Face模型配置所在的目录路径
seed	0	设置全局随机种子
output_dir	'./output'	设置日志、检查点、策略等文件保存路径
load_checkpoint	''	加载权重的文件或文件夹路径
auto_trans_ckpt	False	如果为true，自动转换load_checkpoint以在分布式模型中加载
resume_training	False	启用断点续训功能
run_mode	'finetune'	设置模型的运行模式：train、finetune 或 predict
use_parallel	True	启用并行模式
load_ckpt_format	'safetensors'	加载检查点的格式，可以是 ckpt 或 safetensors

```yaml
use_legacy: False                                          # Specifies whether to use the mcore model.
# pretrained_model_dir: &pretrained_model_dir "path/to/model_dir"  # The directory path where the Hugging Face model configuration is located.
seed: 0                                                    # Set the global seed.
output_dir: './output'                                     # Set the path where log, checkpoint, strategy, etc. files are saved
load_checkpoint: ''                                        # File or folder paths for loading weights
auto_trans_ckpt: False                                     # If true, auto transform load_checkpoint to load in distributed model
resume_training: False                                     # Enable resumable training after breakpoint.
run_mode: 'train'                                          # Set the running mode of the model: `train`, `finetune` or `predict`
use_parallel: True                                         # Enable parallel mode
load_ckpt_format: 'safetensors'                            # The format of loading checkpoint, either `ckpt` or `safetensors`
```

MindSpore Transformers 拉起微调任务
——微调任务

2. 数据集配置修改

配置项	值	说明
train_dataset		
input_columns	["input_ids", "labels", "loss_mask", "position_ids", "attention_mask"]	设置训练数据集的输入数据列
data_loader		
type	HFDataLoader	设置数据加载类
load_func	'load_dataset'	读取数据集时使用的transformers接口
split	"train"	在线数据集的子集名称
path	"llm-wizard/alpaca-gpt4-data"	在线数据集名称
handler		
- type	take	取数量函数
n	2000	取数据集的样本数量
- type	AlpacaInstructDataHandler	设置数据处理类
tokenizer		分词器配置
padding_side	"right"	指定Tokenizer的填充位置
trust_remote_code	False	是否信任远程下载的代码，默认值：False
seq_length	4096	数据集返回的数据序列长度
seed	0	数据集采样的随机种子
num_parallel_workers	8	并行工作进程数量
python_multiprocessing	False	是否启用Python多进程模式以提高数据处理性能
drop_remainder	True	如果最后一批数据样本数少于batch_size，是否丢弃
prefetch_size	1	设置预读取数据量

MindSpore Transformers 拉起微调任务
——微调任务

3. 模型配置修改

1. **自动读取**：自动读取pretrained_model_dir中的config.json文件，找到对应的模型Config和模型类

2. **按需添加**：**新增的MF配置**，或者想要**覆盖config.json的配置**，可以直接在yaml中进行配置。

```python
@register_mf_model_parameter(
    mf_model_kwargs=MFModelConfig(
        pad_token_id=151643,
        block_size=32,
        num_blocks=1024,
        normalization='RMSNorm',
        add_bias_linear=False,
        gated_linear_unit=True,
        use_contiguous_weight_layout_attention=False
    ))
@ignore_and_delete_parameter(extra_ignore_param=[
    ('max_window_layers', NotSupportedInfo.useless),
    ('sliding_window', NotSupportedInfo.useless),
    ('use_sliding_window', NotSupportedInfo.useless),
    ('layer_types', NotSupportedInfo.useless),
])
def __init__(self,
             vocab_size=151936,
             hidden_size=4096,
             intermediate_size=22016,
             num_hidden_layers=32,
             num_attention_heads=32,
```

新增MF配置

删除配置

HFConfig迁移示例

```yaml
model:
  model_config:
    # hf config
    model_type: "qwen3"
    architectures: [ "Qwen3ForCausalLM"]
    vocab_size: 51200                    # Vocabulary size of the model
    hidden_size: 12288                   # Dimension of hidden layer
    intermediate_size: 49152             # Dimension of intermediate layer
    num_hidden_layers: 24                # Number of hidden layers
    num_attention_heads: 96              # Number of attention heads
    # ...
    # hf config
    # 设置后会覆盖config.json中的配置
    vocab_size: 51200                    # Vocabulary size of the model
    # ...
    # mf config
    params_dtype: "float32"
    compute_dtype: "bfloat16"
    layernorm_compute_dtype: "float32"
    softmax_compute_dtype: "float32"
    # ...
```

yaml配置示例

谢谢观看！

MindSpore Transformers LLM训练迁移与实践

MindSpore Transformers 大模型系列课程